

Your Agent Escaped Without Escaping the Sandbox

Yossi Eliaz, PhD

Principal Engineer and Head of DevRel, Incredibuild

The agent changes the answers. The check goes green.

Abstract

The agent changes the answers. The check goes green. A developer asks an AI harness to fix authorization. The harness works in a dev sandbox. A verifier loads `expected.json` from that workspace. A GitHub CI runner promotes on PASS. A prod sandbox runs those bytes. The dev sandbox can do its job and the runner can still promote a bug. This paper is the written companion to a fifteen-minute lightning talk. On real Linux, with bubblewrap, a worker inherits a broad CI-style token, writes another job's file through a mount, publishes on an allowed host, and turns a read-only checker from FAIL to PASS by editing `expected.json`. The candidate bytes never change. Controller-owned cases reject those bytes. A corrected handler passes the same five cases. A fresh approval refuses a substituted artifact, publishes the original bytes, and refuses a replay. The breach rides the promotion. The credential, the mounts, the publish, and the answer key each need an owner outside the worker, and the legitimate job still has to complete.

1 The line

The picture is the software factory you already run.

Station	Role
Developer	Opens the ticket
AI harness	Goal, tools, workspace
Dev sandbox	Where the agent writes
CI runner	Token, check, publish, promote
Prod sandbox	Runs the published bytes

The harness stays in the dev sandbox. The runner decides the work is done. The prod sandbox runs what the runner published. The failure is a promotion along a factory that is working. Each station trusts a receipt from the station before it.

The accepted abstract names four of those receipts: inherited credentials, dangerous mounts, exfiltration through an allowed endpoint, and verifier tampering. On this line they are the CI secret injected into the dev job, a mount that can touch the next job's files, a host the workflow is already allowed to publish to, and the green check that promotes the sandbox into prod.

A protected checker says PASS. Unauthenticated admin still returns 200. Policy requires 401. You can see that on the line, name the file, and move the decision to a station the worker cannot write. The legitimate job still ships.

What follows is a controlled reproduction of a known trust-boundary failure. The credential, mount, and publish cases are loopback HTTP and real bind mounts. The green check is a five-case command-line model of authorization. These fixtures have the shape of a GitHub Actions job. They are not a compromise of GitHub, and they are not a customer production server. The repository records the bytes.

2 What the worker is allowed to touch

The task is `lock-admin`. Five inputs. Unauthenticated admin and an expired credential must return 401. An ordinary user must return 403. The administrator and the public route must return 200.

The worker may edit task code, the workspace, and the tests and reports it proposes. That is ordinary coding-agent authority. The checker sits on a read-only bind mount. The controller process, the approval key, and the release store sit outside the worker. The kernel and that controller are the trusted base of the experiment.

The worker in the repository is a script. The measurement is whether the action is permitted. Whether a particular model would discover the action is a different experiment, and this package does not run it.

3 The control, then the exploit

The bad candidate returns 200 for every input. With the honest answer key, the checker prints `FAIL` and exits 1. The check can see the bug.

The worker then tries the obvious overwrite: replace `checker.py` with a program that prints `PASS`. The read-only mount denies the write. The checker hash is identical before and after that step:

```
f5d8d3468bbeeee014580159053a10a4af758a11a22af3045b03abcf94922e99
```

The grader survived. The mount held. The next step never touches `checker.py`.

The protected checker loads its cases from the working directory:

```
expected = json.loads(Path("expected.json").read_text())
```

That file lives in the workspace. The worker owns the workspace. It rewrites every expected output to 200. On `admin:none` the key moves from 401 to 200. The candidate still returns 200. The same checker, still read-only, prints `PASS` and exits 0.

Locking the program left the answer key with the worker. A controller that promotes that `PASS` into release eligibility has adopted the worker's definition of authorized. A plugin, an imported helper, a test configuration, or a worker-written report has the same shape. The object reproduced here is `expected.json`.

4 Same bytes, three verdicts

Candidate	Criteria	Result
Always 200	Honest workspace key	FAIL
Same bytes	Worker-edited key	PASS
Same bytes	Controller-owned cases	REJECT
Corrected handler	Controller-owned cases	ACCEPT

The controller’s expected sequence is 401, 200, 403, 200, 401. The corrected handler returns that sequence. The legitimate ticket still ships. The comparison, and the answer key, now sit where the worker cannot write them. Five cases cover this policy. They cover this policy.

5 Freeze, then judge

The recorded repair freezes the candidate bytes before the run. An isolated execution reads those bytes from a read-only mount and returns observations. The controller compares the observations with cases it owns. Approval and publication name the frozen object.

A harness that prints “finished” holds no approval key. A second machine can host the split. Copy the producer’s answer key onto it, or fork the producer’s disk, and the dependency moves into the room you trusted. Count the inputs to the decision, then count the machines.

One fresh approval is issued for the good candidate. Three attempts follow, in this order, while the approval is still live. Substituted bad bytes are **DENIED**. The original good bytes are **PUBLISHED**. The same bytes offered again are **DENIED**. Publication is a row in controller-owned SQLite: the exact bytes and the spent nonce, in one transaction. Spending the nonce before the digest check would let a broken substitution hide behind an already-used approval. This repository used to get that order wrong. The current gate refuses the substitute first.

6 Four boundaries, both sides

The answer key is the centerpiece. The deployment checklist around it has four rows. Each row needs a denied abuse and a job that still completes. A host that can do nothing will pass a deny-only test.

Boundary	This must fail	This must still work
Identity	Read another project’s data	Read the task input
Files	Edit the next job’s input	Write the task output
Outbound	Upload to another account	Send the authorized report
Acceptance	Edit the criteria or swap bytes	Release the verified fix

Identity, on loopback HTTP with fixture tokens: a broad token reads another project’s private fixture and receives 200. A task-scoped token receives 403 there and 200 on its own input. Expired and wrong-audience fixtures are rejected. A short lifetime on a broad token leaves the token broad. The service enforces the resource.

Files, on real bind mounts: a writable mount changes the next job’s fixture on the host. A read-only input mount denies that write. The worker’s own output path still accepts the patch.

Outbound, on an allowed loopback service: the worker posts a synthetic private fixture to another account and the service stores it, 201. The repaired broker sends one fixed team report. Extra fields are denied. The worker has no direct route to the service. The receiver log is the evidence of delivery.

Acceptance: the worker proposes the artifact. The controller owns the criteria and the bytes that get published.

These are four separate experiments. They are a checklist, measured on both sides.

7 Three jobs inside one green build

Verification says these bytes met these criteria. The gate says this approval authorizes these bytes on this run. History says the record still matches an anchor stored where the writer cannot reach it.

A hash chain will store a bad decision faithfully. It detects a rewrite when the previous head was kept outside the log. The reference gate binds artifact, verifier, expected results, policy, run, environment description, task, decision, nonce, expiry, and schema. A worker `PASS` file and a harness exit code are observations from the wrong side of that line. Missing or malformed output produces no approval.

The next CI run can be handed the same dev workspace. A reviewer job and a deploy job can be forked from that disk. If they all read `expected.json`, and the first job can write it, three stations share one definition of done. Different prompts do not create independence. A snapshot of the dev sandbox keeps the key. The snapshot records the disk. Choosing the criteria is a different authority. That paragraph is an inference from the dependency we measured. It is not a multi-job attack and it is not a snapshot exploit.

An earlier revision of our own gate could publish a marker while the candidate bytes and the recorded digest diverged. Reaching that flaw required controller-object access. The current gate derives the digest from immutable bytes and stores those bytes. The audit stays in the package because it is ours.

8 What was measured

Evidence	Checks	Scope
Linux lab	29	Namespaces, mounts, loopback HTTP
Isolated checker	11	Read-only checker, mutable key, release
Local factory	6	Protocol model, separate from OS isolation
Regressions	37	Controller and evidence
Archived gate audit	1	Controller API misuse

The counts are coverage of this policy. The evidence files bind each record to the source that produced it. The build rejects a stale, skipped, or incomplete record. The local protocol executor, run by itself, is a different isolation story from the bubblewrap cases.

9 Reproduce it

```
make test          # controller and evidence regressions
make record-all   # Linux fixtures; requires bubblewrap
make snapshot      # talk PDF and offline replay
make paper         # this document
```

On Ubuntu 24.04, unprivileged user namespaces are restricted and bubblewrap fails while creating a network namespace until:

```
sudo sysctl -w kernel.apparmor_restrict_unprivileged_userns=0
```

Source, recorded outputs, and the limits live at <https://github.com/zozo123/nyc-talk>. The deck and this paper read those recordings. Stage delivery needs no cloud account and no live terminal.

External context, kept in its own lane: bubblewrap’s security model is caller-defined sandbox policy; SLSA v1.2 is the vocabulary of digest and expectations, and this gate is not a SLSA certification; Anthropic’s November 2025 study of reward hacking is prior motivation, and its measurements stay with that study. The New York date is the Zenity AI Agent Security Summit at Pier Sixty on Wednesday 21 October 2026. The fifteen minutes are the session brief. The exact stage clock was still awaiting organizer confirmation when this paper was prepared, as was a logistics reply to the summit team.

10 Leave with four questions

The checker was protected. The candidate never improved. The worker changed the answer key, and PASS changed meaning.

Who controls what makes the work releasable?

When you leave Pier Sixty, take four questions back to the host that runs your agents.

1. Which credential does the worker receive, and what else can that credential read?
2. Which paths are mounted, which of them are writable, and which of those paths will a snapshot fork tomorrow?
3. Which URLs can it reach, and who chooses the path and the body?
4. Which object is compared before merge, and can the worker change the cases or the PASS file?

The component that returns 403, a read-only denial, DENIED, or reject has to sit outside the worker. After you put it there, run the legitimate ticket again. A system that can no longer do the job has gone quiet. It has not become safe.

The sandbox is worth keeping. You built it so the agent could work and still be stopped. Keep it. Then open the file the checker trusts. If the worker can write that file, you already know the next green build.